CSA1717-ARTIFICIAL INTELLIGENCE

CO2-AT3-DRY RUN TEST

REPORT

Name: Roshitha **Reg No:** 192424400

Question 1: A* Search Algorithm

Title: Implementation and Dry Run of the A* Search Algorithm using Parent-Pointer Path Reconstruction

Aim

To implement the A* Search Algorithm to determine the shortest path between a start node and a goal node using heuristic values.

Objective

- To understand how the evaluation function f(n) = g(n) + h(n) drives node selection in informed search.
- To trace how g(n) values are updated when a cheaper route to an already-discovered node is found.
- To reconstruct the optimal path using parent pointers rather than storing full paths in the priority queue.
- To determine the shortest path and total path cost from node A to node G.

Problem Statement

Given a weighted graph and heuristic values for each node, implement the A* Search Algorithm to find the shortest path from node A to node G. Display the current node, open list, closed list, g(n), h(n), and f(n) at every iteration, and determine the optimal path and total path cost.

Algorithm

1. Push the start node onto a priority queue (Open List) with f(n) = h(start), g(start) = 0.

2. Pop the node with the smallest f(n) from the Open List; skip it if already in the Closed List.

3. If the popped node is the goal, reconstruct the path via parent pointers and stop.

4. Otherwise, move the node to the Closed List and examine each neighbour not already closed.

5. Compute the tentative g(n) for each neighbour; if it improves on any previously recorded cost, update g(n), set the parent pointer, and push the neighbour with its new f(n).

6. Repeat from step 2 until the goal is popped or the Open List is exhausted.

Input

Graph with weighted edges (A, B, C, D, E, G); heuristic values h(n) for each node; Start Node = A; Goal Node = G.

Sample Output

Optimal Path: A -> B -> E -> D -> G

Total Path Cost: 8

Result

The A* Search Algorithm successfully found the optimal path from A to G as A → B → E → D → G with a minimum total path cost of 8, updating D's parent from B to E mid-search once a cheaper route was discovered, confirming the algorithm correctly revises earlier decisions when better-cost paths are found.

Conclusion

Tracking g(n) per node with parent pointers, rather than carrying whole paths inside the priority queue, makes it straightforward to see exactly when and why a node's best-known route changes during the search, while still guaranteeing the same optimal result as long as the heuristic remains admissible.

Question 2: Minimax Algorithm with Alpha-Beta Pruning

Title: Implementation and Dry Run of a Recursive Minimax Function with Alpha-Beta Pruning

Aim

To implement the Minimax Algorithm with Alpha-Beta Pruning, using a single recursive function that can evaluate MAX and MIN levels generically, for selecting the optimal move in a two-player game tree.

Objective

- To understand how alpha and beta bounds propagate between MAX and MIN levels during recursion.
- To apply the beta ≤ alpha pruning condition to skip evaluating leaves that cannot affect the outcome.
- To determine the best move and final minimax value for the MAX player.
- To identify which leaf node(s) are pruned during the search.

Problem Statement

Given a game tree with MAX and MIN levels, implement the Minimax Algorithm using Alpha-Beta Pruning. Evaluate the tree from left to right while displaying Alpha (α), Beta (β), selected values, and pruned nodes. Finally, determine the best move and the final minimax value.

Algorithm

7. Start at the root MAX node with alpha = -∞ and beta = +∞.
8. Recurse into the left MIN subtree, updating beta after evaluating each leaf in order.
9. Return the minimum leaf value found to the MAX node and update alpha with it.
10. Recurse into the right MIN subtree with the updated alpha.

11. After each leaf, check whether beta ≤ alpha; if so, stop evaluating the remaining leaves of that MIN node (they are pruned).

12. Return the selected value from each MIN node to the MAX node.

13. MAX selects the larger of the two returned values as the final minimax value and reports which subtree it came from.

Input

Six leaf node values, entered left to right.

Sample Input

Leaf 1: 3

Leaf 2: 5

Leaf 3: 6

Leaf 4: 9

Leaf 5: 1

Leaf 6: 2

Sample Output

Left MIN Value: 3

Right MIN Value: 1

Final Minimax Value: 3

Best Move: Left Subtree

Pruned Node: 2

Result

The recursive Minimax function with Alpha-Beta Pruning correctly determined a final minimax value of 3 with the best move being the left subtree, pruning the rightmost leaf (value 2) of the right MIN node since beta (1) had already fallen to or below alpha (3), meaning that leaf could never have changed the outcome.

Conclusion

Implementing Minimax as a single recursive routine rather than two hand-written MIN-node loops generalises cleanly to deeper or wider trees, while the alpha-beta bounds still eliminate exactly the branches that cannot influence the final decision, giving the same optimal result with fewer node evaluations.